Porting a rigid-certificate method to BB(6)

Two censuses of the 1,064-machine holdout list, a negative result about what boundary rigidity buys, three cryptid candidates, and a Lean formalisation of the machinery

Summary

A method that decided nine FRACTRAN holdouts was ported to the BB(6) Turing-machine holdout list. This report records what transferred, what did not, and why. Four results are worth stating up front.

1. A negative that transfers. Rigid phase boundaries do not imply a rigid phase word. In the FRACTRAN setting the two came together and fitting the boundary was the hard part; on Turing machines they come apart completely. Four machines were found whose boundary families are exactly affine and confirmed at a hundred times the fitting budget, and none of them is decidable by the method, because the word between boundaries never compresses at any block size. A rigidity census therefore over-counts what a certificate method can decide.

2. Three cryptid candidates. Of 1,064 machines, five have a two-level structure whose outer map meets the cryptid criteria; three survive deeper runs. Their halting questions reduce to orbit avoidance for explicit expanding integer maps with multiple branches and no periodic branch pattern over the observed range.

3. An acceleration of about 158 orders of magnitude. Recognising the machines' induction rule at run time takes the reachable horizon from roughly 10^7 base steps to roughly 10^{165} in 45 seconds, and the observable outer orbits from 10–12 steps to 289–294.

4. The halting criterion, and a Lean formalisation of the machinery. All three candidates halt if and only if a two-state scanning gadget meets two adjacent zeros. The Turing-machine semantics, the chain lemma, the block-crossing table and the halting criterion are formalised in Lean 4, mathlib-free, with no `sorry` and no added axioms.

Nothing here decides whether any of the three machines halts. That is the open problem they exist to pose, and no claim is made against it.

1. Setting

The Busy Beaver challenge maintains lists of small Turing machines whose halting is undecided by any deployed decider. The BB(6) list used here has 1,064 machines up to equivalence (mxdys, 28 July 2026). These machines have survived Cyclers, Translated Cyclers, Backward Reasoning, Closed Tape Language, n-gram CPS, WFAR and RepWL, so any method that reports a large fraction of them as easy is reporting a bug.

That observation is used throughout as a calibration test. The first version of the rigidity detector classified about 15% of the list as having polynomial phase structure, which is impossible; the cause was a missing filter, described in section 3.

A *cryptid*, in this community's usage, is a small explicit machine whose halting is provably equivalent to an open Collatz-type orbit-avoidance problem. Three structural features make such problems hard, and they are what section 5 measures: the dynamics are piecewise affine, the map expands on average, and the branch taken depends on ever deeper digits of the argument.

2. Acceleration is the whole game

A cell-at-a-time simulator run for two million steps on one of these machines sees the head visit about fifty new cells. Fifty. Whatever structure they have lives at a scale a raw simulator does not reach, so every later section depends on compressing the simulation exactly.

Three layers were built, each cross-checked step-for-step against the one below it. Correctness here is load-bearing: a plausible acceleration with a wrong step count corrupts every downstream number.

layer	what it does	verification
`tm.py`	cell at a time	all four Busy Beaver champions exact; BB(5) at 47,176,870 steps and 4,098 ones
`blocktape.py`	chain steps	step-exact against tm.py on 3,000 random machines and 60 holdouts, tapes identical
`macro.py`	macro machines, block sizes 1 to 6	3,454 machine/block-size pairs cross-checked

Figure 1. The simulator stack. Each layer is checked against an independent, simpler implementation rather than against itself.

A measured negative worth recording. Chain steps alone — crossing a run of identical cells when a state re-enters itself in the same direction — give a speed-up of about 1.2 times on this list. These machines have almost no base-level self-loops; they bounce. Grouping cells into blocks of size b and treating each block as one symbol (Marxen–Buntrock macro machines) creates the self-loops that do not exist at cell level, and the speed-up on BB(5) becomes about 1,900 times.

Two decisions come free from simulating inside a block. If the head never leaves the block within the pigeonhole bound $|Q| \times b \times 2^b$, a configuration must repeat and the machine never halts; if it halts inside, the machine halts. Both are proofs, not heuristics.

3. First census: single-level rigidity

A machine is *rigid* when some sub-sequence of its configurations — its phase boundaries — has coordinates given by formulas in EXP = $\{a + bn + c \cdot q^n\}$, with the same word of machine steps between consecutive boundaries. A Turing machine has no coordinate vector until the tape is run-length encoded, at which point block lengths are the coordinates and the definition transfers unchanged.

class	count	share
NONRIGID	626	58.8%
FEWPHASE	431	40.5%
GEO	4	0.4%
UNCONFIRMED	3	0.3%

Figure 2. All 1,064 machines, 3,191 seconds.

Two filters are what make these numbers mean anything.

Eventual positivity. A fitted counter with a negative trend describes a transient, not a phase family. One real example from the sweep: a counter fitted as 381 − 4n, exact over all 96 phases it was fitted on, and then the family simply ends because the counter reaches zero. The longer the transient, the more convincing the fit looks. Without this filter the detector reported the impossible 15% POLY rate.

Confirmation. Every fit is determined by three phases and then re-tested on phases it never saw. Fits that fail are counted separately as UNCONFIRMED rather than silently dropped.

4. Rigid boundaries do not imply a rigid phase word

The four GEO machines (lines 360, 833, 852, 1005) have genuinely rigid boundaries. Re-run at a 20M macro budget, a hundred times the budget their formulas were fitted at, each reproduced seven or eight unseen phases with zero mismatches. For line 360 the boundary is

```
B(n) = 1^(6+2n) 0 1^23 0 1^12, head at far left facing left, state D
steps between boundaries = 72140 − 6n + 96 · 2^n
```

— a linear tape with an exponential clock. But a certificate needs the word *between* boundaries to be a fixed list of stages, and it is not.

block size	phase-word lengths	stages	max run
1	15, 30, 60, 120, 240	same as length	1
2	11, 19, 39, 78, 156	same as length	1
4	29, 141, 568, 2180, 8708	same as length	1

Figure 3. The phase word never compresses: every run-length count is 1, at every block size tried.

There is real structure, uniform across all four machines and every level: $|W(n+1)| = 2|W(n)|$ exactly, and the words agree on a common prefix of exactly $|W(n)| - 6$ symbols — the same constant 6 for all four, including the machine whose base word is 17 rather than 15. The obvious recursion this suggests, $W(n+1) = W(n)[:-6] + X + W(n)$ with X the constant divergence block, is false at every level; it was tested. The divergence from "W(n) repeated twice" grows proportionally (13, 22, 44, 88 symbols), so it is not a bounded-edit substitution either.

The conceptual point. For the nine FRACTRAN holdouts, boundary rigidity and phase-word rigidity came together, and fitting the boundary was the hard part. On Turing machines they come apart: the boundary family can be exactly affine while the word between boundaries is exponentially complex and incompressible. The boundary fit is necessary and nowhere near sufficient. Here four candidates yielded zero decisions.

Deciding those four needs a decider for the reachable *set* — a closed-tape-language argument — not a certificate for a single orbit. That is a different tool, and one the community already has stronger versions of.

5. Second census: cryptid-shaped outer maps

A two-level machine has an inner loop that iterates an affine map and an outer map that carries one reservoir value to the next. The outer map is the object the cryptid criteria apply to. The detector was calibrated on the BB(6) machine decided in April 2026 via Baker–Wüstholz, whose inner recurrence it recovers as $x \to 3x + 4$ and which it classifies as cryptid-shaped.

class	count	share
no two-level structure	1,033	97.1%
INSUFFICIENT	22	2.1%
CRYPTID-SHAPED	5	0.5%
NOT-EXPANDING	3	0.3%
PREDICTABLE-BRANCHES	1	0.1%

Figure 4. All 1,064 machines, 523 seconds.

One criterion runs the opposite way to intuition. Piecewise affineness is not something to look for: the outer map of a two-level machine is affine on each branch by construction, since it is built from macro rules that are themselves affine. What must be tested is whether a *single* affine branch explains the whole orbit. If one does, the orbit has a closed form and the machine is tractable — which is what a cryptid is not. A failed global affine fit is evidence of several branches, hence evidence *for* the cryptid shape. Getting this backwards initially made the Baker–Wüstholz machine read as having no closed form rather than as cryptid-shaped.

Cryptid-shaped does not mean undecided. The Baker–Wüstholz machine is cryptid-shaped and was decided, with heavy machinery. The label says where the difficulty lives.

5.1 Two corrections found by running deeper

The last outer step is systematically truncated. The simulation stops at a fixed macro budget, which lands in the middle of the final inner loop, so that loop's length is an undercount. Raising the budget from 8M to 40M changed one machine's last branch index from 9 to 10 and flipped its verdict. A criterion that depends on the final delta is reading the budget, not the machine; the last step is now dropped.

A periodic branch pattern is as predictable as a constant one. The delta sequence 1, 2, 3, 1, 2, 3 is generated by a three-state automaton, so a bounded invariant does track the branch sequence. Testing only for constant deltas let one machine through as cryptid-shaped.

Both corrections removed candidates. The surviving three:

line	inner	outer steps	branch deltas	growth	verdict
336	`2x+4`	10	`2,0,2,2,2,2,1,1`	2.94	CRYPTID-SHAPED
555	`2x+5`	12	`1,1,1,0,2,1,1,2,2,1,2`	2.56	CRYPTID-SHAPED
1002	`2x+4`	10	`2,2,1,1,1,2,2,2`	2.93	CRYPTID-SHAPED
106	`2x+3`	6	`1,1,1,1,1 (period 1)`	2.06	predictable
990	`2x+2`	7	`1,2,3,1,2,3 (period 3)`	4.48	predictable

Figure 5. All inner recurrences are base 2, where the Baker–Wüstholz machine is base 3 — a different family.

The three machines, in the standard format:

```
336  1RB0LD_1LC0RA_1RA1LB_1LA1LE_1RF0LC_---0RE
555  1RB1RE_1LC0RA_1RD0LB_1LB1RC_1LF0RD_---0LE
1002 1RB1LC_1RC0LD_1LA0RB_1LB1LE_1RF0LA_---0RE
```

6. The unit lemma

The inner loop of each machine is a fixed unit of five macro steps. Observed first as an empirical regularity, it was then derived symbolically, and the two routes agree.

The symbolic route needed one correction. A symbolic macro simulator, carrying block counts as expressions rather than integers, stalls after five to seven steps: it refuses to consume a single cell from a symbolic block, because for some values of the parameter the block survives and is read again and for others it vanishes and the next block is read. Those are different runs, and an engine that picks one is simulating a branch rather than the machine. The stall is sound and was mistaken for a wall. What the lemma wants is the guard carried forward, not the run stopped — but only if the block being lifted is the one the unit actually sweeps. Lifting the wrong block, a constant-length one, fragments the tape into a configuration the machine never reaches; the first attempt did exactly that.

line	one unit	cost
336	`(1, c1, x, c3) → (1, c1−1, x+2, c3−1)`	`4x + 12`
555	`(x, c1, 1, c3) → (x+2, c1−1, 1, c3−1)`	`4x + 4`
1002	`(1, c1, x, c3) → (1, c1−1, x+2, c3−1)`	`4x + 12`

Figure 6. The unit, crossed symbolically with the correct block lifted.

The derivation reproduces the measured cost law from an independent route. At unit j the swept block has $x = x_0 + 2j$, so $4x + 12 = 8j + (4x_0 + 12)$, which at $x_0 = 1$ is $8j + 16$ — exactly the law measured empirically over 151 and 156 consecutive units. Checked additionally at x = 1, 2, 3, 7, 11, 20, 53, 100, 301 and 1000: ten out of ten exact on all three machines.

Stated at cell level for line 336, and verified on 54 independent instances (54 out of 54 exact, with arbitrary surrounding tape):

```
for all x, a, b and arbitrary Lr, Rr:

  [11] (10)^(a+1) Lr | (10)^x (11)^(b+1) Rr  state A, facing left
    -- 4x + 12 steps -->
  [11] (10)^a Lr     | (10)^(x+2) (11)^b Rr  state A, facing left
```

Traced at x = 3 and x = 5, the run decomposes into four pieces, two of them chains:

piece	steps	x=3	x=5
prologue, fixed	4	4	4
chain right: x+2 crossings of 01 → 10	2x+4	10	14
turnaround, fixed	2	2	2
chain left: x+1 crossings of 10 → 01	2x+2	8	12
total	4x+12	24	32

Figure 7. Both chains are instances of block crossings already proved in Lean, so the remaining proof is composition.

7. Rule-based acceleration

A simulator that recognises the unit at run time need not walk it. At each macro step the accelerator looks ahead for a return to the same skeleton, state and direction; if the counters move by the same fixed vector on two consecutive repetitions and the per-repetition cost is constant or grows by a constant, it computes how many repetitions fit before a guard would fail and jumps the whole way. The number of repetitions is chosen one short of any guard failing, so a jump never crosses a branch. That jump is the composition of prologue, n units and epilogue — the closed form, applied.

Verified against the unaccelerated simulator at 5.0, 6.3 and 5.1 × 10^7 base steps for the three machines: identical skeleton, identical counters, identical base-step count, with 63, 90 and 79 jumps skipping 12,409, 15,215 and 14,956 units. Beyond that range the detector still self-checks, observing two full repetitions with matching deltas before every jump, but that is a safeguard rather than a proof.

line	outer steps	growth per step	branch index range	delta period	halted
336	289	2.4648	3 to 378	none	no
555	294	2.4166	3 to 374	none	no
1002	290	2.4936	3 to 384	none	no

Figure 8. Reach goes from about 10^7 base steps to about 10^{165} in 45 seconds, roughly 158 orders of magnitude, and outer orbits from 10–12 steps to 289–294. No halt occurs anywhere in that range.

At about 290 outer steps the absence of a periodic branch pattern is on much firmer ground than it was at ten, and the growth rate is stable rather than a small-sample artefact.

8. The halting criterion

All three machines have exactly one undefined transition, state F on symbol 0, and in all three F is entered from exactly one place: state E reading 0. E and F therefore form a scanning pair. From E on a 0 the machine writes 1, steps one cell in the scan direction and lands in F; F on a 1 writes 0, steps again and returns to E; F on a 0 halts; E on a 1 leaves the scan entirely.

```
HALT if and only if, at some moment, the machine is in state E reading 0 with the next cell
in the scan direction also 0.
```

Equivalently the pair consumes the word 01 (mirrored to 10 for line 555) repeatedly, and halts on 00. Over 6×10^6 base steps per machine there are 3,005, 4,130 and 4,904 such scans and not one carries a 00; every one reads 01 and continues.

This is the reduction the whole exercise was for. Halting is now a condition on the tape word at a specific, identifiable moment rather than a statement about the machine's entire future.

9. Lean formalisation

591 lines, Lean 4.32.2, mathlib-free, no `sorry`, no `native_decide`, no added axioms — results depend on `propext` and `Quot.sound` only. Thirty-six `#guard` checks are evaluated at compile time.

The representation is the proof strategy. The first version put the head on a cell and proved a sweep lemma for a state re-entering itself in the same direction. It compiled, and it was useless: none of the three machines has such a transition at cell level — which is exactly why the Python side needed macro machines in the first place. The right primitive is a block crossing, and with the head *between* cells a block crossing is literal list concatenation, so the induction goes through with no index arithmetic. With the head on a cell it does not.

result	content
`Steps.trans`	runs compose, step counts add; depends on no axioms at all
`crossR_rep, crossL_rep`	the chain lemma: n copies of a block are crossed in n·k steps, arbitrary machine, arbitrary block
`steps_of_runFor`	bridge from the executable runner to Steps, so the kernel computes intermediate configurations
`24 crossings`	the chain-step table, generated from the same code the measurements used, each with an executable check
`m336/m555/m1002_halt_iff`	each machine halts iff state F reads 0 — the E/F gadget, formal

Figure 9. What is proved.

The semantics are pinned against ground truth at compile time: `#guard` evaluates BB(2), BB(3) and BB(4) to 6, 21 and 107 steps with 4, 5 and 13 ones. A drift in the step convention — which cell is written, whether the halting transition counts, what unwritten tape reads — breaks the build. Further guards check that none of the three candidates halts within 20,000 steps, which would catch a transcription slip in the transition tables.

One subtlety is load-bearing. The halt lemmas carry a hypothesis that the state index is below 6, because the table lookup also returns nothing for an out-of-range state; without it the statement is false. Reachable states are always in range, but that is a fact to be carried rather than assumed.

10. What is and is not established

Established. Both censuses, on the full list, with the calibration and confirmation filters described. The negative result of section 4, from direct measurement at every block size tried. The unit lemma of section 6, derived symbolically and agreeing with an independent empirical measurement to the constant. The acceleration of section 7, cross-checked exactly against the unaccelerated simulator over the range where that is feasible. The halting criterion of section 8, read from the transition tables and confirmed over millions of steps. The Lean development of section 9.

Not established. Whether any of the three machines halts. That is the open problem, and these results sharpen its statement without touching it.

The equivalence itself — a machine-checked proof that a given machine follows a given map, and halts if and only if that map's orbit meets an explicit set — is not finished. Its remaining obligations are: composing the four fragments of Figure 7 into the unit lemma in Lean, iterating it, composing to obtain the outer map, and

expressing the section-8 condition in terms of the section counters. Every one of those is assembly on results already stated and verified; none is open. That distinction is the honest characterisation of where this stands.

Two limits are worth naming precisely. The absence of a periodic branch pattern is measured over 289 to 294 outer steps, not proved; a longer period could hide beyond that range. And the cost wall is real: growth is 2.5 to 2.9 per outer step with work scaling in the reservoir value, so each further outer step costs roughly three times the last, and reaching twenty more is a factor of about 3^{20}.

11. Reproducing this

All code, logs and the Lean development are at github.com/TomZahavy/collatz-cryptids, under `collatz/bb6/`. The censuses are `sweep.py` and `sweep_cryptid.py` over `bbf/bb6_holdouts_1064.txt`; every simulator has a self-test that runs its own cross-checks; `lake build` in `bb6/lean` reproduces the formalisation in a few seconds. Detailed measurements, including the failed hypotheses, are in `bb6/RESULTS.md`.

The failed hypotheses are recorded deliberately. Three of the corrections in this report — the missing positivity filter, the truncated final outer step, and the periodic-branch blind spot — were mistakes that produced confident wrong answers before they were caught, and each was caught by a check that could have been run earlier.